A direct enumeration of permutations sortable by a generalized (2,2)-stack

Columbus Labs
info@columbus-labs.com

12 October 2026

© 2026 Columbus Labs. Licensed under CC BY 4.0.

Abstract

A generalized $(2, 2)$-stack permits insertion and removal at either of its top two positions. We derive its known permutation generating function directly from these operations. Atkinson's canonical sorting rule gives unique histories. Recording the comparisons imposed by their pushes produces ordered forests; retaining only leaf and nonleaf marks gives an exact finite-control pushdown model. A first-return decomposition reduces its enumeration to two series and yields

$$2xf^3 - (2x + 3)f^2 - (x - 7)f - 4 = 0, \qquad f(0) = 1.$$

The polynomial is not new. The purpose is the mechanism-based derivation asked for in Vatter's Problem 5.11. The proof does not use the forbidden-pattern basis or infer an equation from computed coefficients.

1 The result and its provenance

Write a stack from top to bottom. Its positions are numbered $0, 1, \ldots$. A push inserts the next unread input entry in position 0 or 1; an empty stack allows only position 0. A pop removes an existing entry in position 0 or 1. An input permutation is **sortable** if some sequence of these operations outputs $1, 2, \ldots, n$. The stack has no capacity bound.

Let a_n count sortable input permutations of length n, with $a_0 = 1$, and let $f(x) = \sum_{n\geq 0} a_n x^n$.

Theorem 1 (known enumeration, with a mechanism proof below). The series f is the unique formal power series with constant term 1 satisfying

$$2xf^3 - (2x + 3)f^2 - (x - 7)f - 4 = 0.$$

Atkinson [1, Theorem 4 and its proof, p. 7 of the author manuscript] established the canonical sorting rule used below and the avoidance basis. His conjectured parametric form appears on p. 8 of that manuscript. Vatter [2, Section 5] reports that Atkinson's conjectured enumeration was subsequently proved by Jay Pantone using Combinatorial Exploration from that basis, with the proof credited there as personal communication. Vatter's Problem 5.11 asks for a derivation from the sorting mechanism. The same counting sequence and cubic appear in OEIS A393395 [4]. We therefore claim neither the polynomial nor the canonical sorter as new. Canonical operation histories

also occur in the restricted container framework of Albert, Homberger, Pantone, Shar, and Vatter [3]; its leftmost-pop argument does not directly cover the present two-pop machine. In particular, pop/bypass normalization and a pop-permission flag are prior methodology [3, Sections 1.4 and 2.1]. Canonical languages and pushdown enumeration also appear in the forklift work of Albert and Atkinson [5]; explicit first-exposure grammars and elimination appear in Elder, Lee, and Rechnitzer [6]. These are methodological antecedents. The forklift machine transfers blocks, and [6] treats two stacks in series; neither is the top-two-position machine here. The present argument concerns its comparison forest, exact leaf-word representation, and resulting direct enumeration.

We first choose canonical histories, encode their comparisons by ordered forests, retain leaf and nonleaf marks, and derive a finite first-return system.

All series below are formal. The finite checks summarized at the end validate implementations; they are not steps in the proof.

2 The canonical sorter

The following argument reproves the two canonicalization principles in Atkinson [1, proof of Theorem 4, p. 7 of the author manuscript], using an explicit top-first convention.

Lemma 2 (eager output). If the next required output is accessible and the configuration has a successful continuation, it has one that emits that entry immediately.

Proof. Before emitting that entry m, the original continuation can only push: no other output may precede m. Its depth cannot decrease during these pushes; since its eventual removal uses slot at most 1, its depth is at most 1 throughout. Remove m now and simulate those pushes with m omitted. An insertion in slot $j \in \{0, 1\}$ moves to slot $j - 1$ if m preceded its insertion point, and otherwise remains in slot j. Each adjusted slot is legal. Omit the later pop of m. At that point the stack and unread input coincide with the original continuation, which can be resumed. □

Lemma 3 (top-pair exchange). For $x < y$, if the configuration (y, x, R) has a successful continuation, then (x, y, R) does too, with the same unread input and next-output requirement.

Proof. Follow a successful continuation until x is output. The entry y remains above x throughout. Every token pushed before this output is also above x, since insertion is in the top two positions. Each such token must be smaller than x: otherwise it and y would both remain above x and prevent its removal. Consequently all these new tokens are output before x. No entry of R can be output first, since both x and y block it.

Simulate each intermediate stack by deleting y and reinserting it just below x. Deleting y can only decrease the slot of a push or pop above x; for insertion, subtract one exactly when y preceded the insertion point. Reinserting y below x does not affect those slots. Thus the simulation is legal. Immediately before x is emitted, the two stacks are (y, x, R) and (x, y, R); popping x leaves (y, R) in both, with equal unread inputs and output prefixes. Resume the original continuation. □

Proposition 4 (Atkinson's canonical rule). A permutation is sortable exactly when the following deterministic algorithm succeeds:

1. Pop the next required output whenever it is accessible.
2. Otherwise read the next input, placing the smaller of it and the old top above the larger. If the stack is empty, use its sole insertion slot.
3. Reject if neither operation is possible.

Proof. Maintain the existence of a successful continuation. Lemma 2 justifies the first rule. When the next output is inaccessible, a successful continuation must push. Its two possible placements differ only by their top pair, so Lemma 3 justifies choosing the smaller entry above the larger. Every move consumes or outputs one token; the algorithm cannot continue indefinitely. It therefore accepts every sortable input. Conversely, an accepted run is itself a legal sorting history. Determinism gives uniqueness. □

The explicit inequality matters: after a canonical push, the top two entries increase when read top first. Reversing this inequality fails on the sortable input 2341. It should not be inferred from an orientation-dependent phrase in an older stack drawing.

3 From canonical histories to forests

Write P_0, P_1 for the two pushes and O_0, O_1 for the two pops. Compress each consecutive P_0O_0 pair that introduces and immediately emits the same token into a **bypass** B.

Lemma 5. A successful canonical history has no other immediate push-pop pair. Its compression has no adjacent P_jO_k.

Proof. A P_0O_1 or P_1O_1 pair would emit a larger entry while a smaller one remains. A P_1O_0 pair would delay an entry that was accessible before the push, contrary to eager output. The remaining possibility is P_0O_0. Replacing such a pair by B cannot create an adjacent push-pop pair across that site. □

Represent a tree by a root and an ordered list of child trees; a leaf has no children. A forest is an ordered list of trees, read in preorder. Start with an empty forest and a flag $c = 1$. Introduce one token for each push or bypass. Every parent-child edge requires its parent to be output before its child. The operations are:

- B: introduce and output its token; leave the forest unchanged.
- P_0 on the empty forest: create one leaf.
- P_0 otherwise: wrap the first tree in a new root.
- P_1 on a nonempty forest: insert a new leaf as the first child of its first root.
- O_0: remove the first root and splice its children before the remaining forest.
- O_1: require that the first tree is a leaf and a second tree exists; remove the second root and splice its children after the first leaf.

A push sets $c = 0$. A bypass or pop sets $c = 1$, and pops require $c = 1$. Thus the flag excludes adjacent nonbypass pushes and pops. A complete path begins and ends with empty forest.

Lemma 6. Forest preorder is the stack order, and these operations record exactly the comparison obligations of canonical pushes.

Proof. A P_0 places its new smaller token before the old top: a new root with the old first tree as child records that comparison. A P_1 places a larger new token second: making it the first leaf child records the opposite comparison. The first preorder node is a root. If its tree is nontrivial, the second node is a child and cannot be output first. If that tree is a leaf, the second node is the next root and can be output. Removing either root satisfies its outgoing obligations without altering those between remaining nodes. □

Proposition 7. Complete forest paths are in size-preserving bijection with sortable input permutations.

Proof. Run the unique canonical sorter, compress its bypasses, and record push comparisons as in Lemma 6. Increasing output respects every comparison, and Lemma 5 supplies the flag rule, giving a forest path.

Conversely distinguish tokens by introduction order along a complete forest path. Assign value 1 to its first output token, 2 to its second, and so on; read the assigned values in introduction order to form an input permutation. Expand each B into P_0O_0. Lemma 6 yields a legal stack history with increasing output. Its edge obligations ensure every nonbypass push chooses the canonical slot. A bypass does too: its next-output token is smaller than all currently held tokens.

To check eager output, suppose a push delays an already accessible next-output token t. Until t is emitted, no different token can be output. In the compressed path there must therefore be a nonempty run of pushes followed by a pop of t, with no intervening bypass. This violates the flag rule. Hence the expanded history is exactly the deterministic canonical history. No additional immediate P_0O_0 pairs appear on expansion: a nonbypass push cannot be followed by an explicit pop, and a following bypass starts with another push. Recompression therefore recovers exactly the same bypass sites. The two maps preserve token introduction, output, and operations and are inverse. □

The forest construction handles the extra two-pop comparisons. The underlying pop/bypass normalization and permission flag have precedents in [3].

This step is essential: arbitrary legal sorting histories have multiplicity. Already at length two, the two input permutations have five histories in total. Counting them without canonicalization would give the wrong objects.

4 An exact word representation

Mark a forest node L if it is a leaf and R otherwise; read the marks in preorder. The flag remains unchanged. Writing $a \in \{L, R\}$, the word rules are

Operation	Input word	Output word
B	w	w
P_0	empty	L
P_0	nonempty w	Rw
P_1	av	RLv
O_0	av	v
O_1	Lav	Lv

The pop flag conditions from Section 3 still apply.

Lemma 8. This word model has exactly the same operation paths from empty state as the forest model.

Proof. A push changes only the newly introduced token and, for P_1, the first root's leaf status. Every pop removes a root; no surviving node loses a child, so its mark remains valid. In particular O_1 is available exactly when the first tree is a leaf and another node follows. Both move availability and successor marks are determined by the word and flag. Inductively, a labelled word path uniquely lifts from the empty state to a forest path with the same operations, and projection is its inverse. □

The word need not determine a forest shape. What it determines, with the flag, is which labelled operations are available; their sequence lifts uniquely from the designated empty state.

At a leading L, P_0 and P_1 have the same resulting word. They remain different edges: identifying them would destroy the bijection and lose a factor of two below.

5 First-return equations

For a nonempty word, store its first letter a in finite control and its remaining letters on an ordinary suffix stack. The flag is also finite control. A push sets control to R, disables pops, and pushes onto the suffix either the old control letter (P_0) or L (P_1). A bypass has weight x and enables pops. An O_0 pops the first suffix letter into control; an O_1 from control L discards that suffix letter. Both enable pops. Each push also has weight x, while pops have weight 1.

Fix a suffix beginning with $b \in \{L, R\}$ and an arbitrary tail below it. A **first downstep** stops the first time suffix height falls below its starting height. At that point b has just been popped; the fixed tail has not been touched. Let $T_b(a, d)$ count these paths starting with pops enabled and ending in control d, and let $U_b(a, d)$ count them starting with pops disabled. Rows and columns are ordered L, R. Every final flag is enabled. The immediate terminating-pop matrices are

$$D_L = \begin{pmatrix} 2 & 0 \\ 1 & 0 \end{pmatrix}, \qquad D_R = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

For control L above suffix letter L, O_0 and O_1 leave the same control but remove different tokens. This explains the entry 2.

Let $V(a, e)$ count an initial push and the following first return to the old suffix height, ending in control e. The new control is R with pops disabled, so

$$V(L, e) = 2xU_L(R, e), \qquad V(R, e) = x\big(U_R(R, e) + U_L(R, e)\big).$$

A first-downstep path begins with an immediate terminating pop, a bypass, or a push followed by its uniquely determined first return. Matrix products compose control states in chronological order. Therefore

$$T_b = D_b + xT_b + VT_b, \qquad U_b = xT_b + VT_b = T_b - D_b.$$

Set $K = ((1 - x)I - V)^{-1}$. Since $U_b(0) = 0$, the series V has zero constant term and this inverse exists. We have

$$T_b = KD_b, \qquad U_b = (K - I)D_b.$$

These mutually defined series are unique: their constants are $T_b(0) = D_b$ and $U_b(0) = 0$; the coefficient of degree n in V uses only degree $n - 1$ of U_L, U_R. In the equations for U_b, all remaining contributions use lower degrees of T_b, aside from this already-determined coefficient of V times D_b. Induction on degree determines the entire system. The path interpretation supplies existence. A fixed input degree has only finitely many paths, because every push or bypass introduces a token and pops remove tokens; there are no zero-weight cycles.

Write the second row of $K - I$ as $(u, l - 1)$. Multiplication by D_L, D_R gives

$$V = \begin{pmatrix} 2x(2u + l - 1) & 0 \\ x(3u + l - 1) & x(u + l - 1) \end{pmatrix}.$$

Thus K is lower triangular; write

$$K = \begin{pmatrix} k & 0 \\ u & l \end{pmatrix}.$$

Inverting the triangular matrix yields

$$k = \frac{1}{1 + x - 4xu - 2xl}, \qquad l = \frac{1}{1 - xu - xl}, \qquad u = x(3u + l - 1)kl. \tag{1}$$

The constants are $k(0) = l(0) = 1$ and $u(0) = 0$. Subtraction of the reciprocal identities gives

$$k - l = kl\left(\frac{1}{l} - \frac{1}{k}\right) = xkl(3u + l - 1) = u.$$

Consequently $k = u + l$, and the second identity in (1) becomes

$$l = 1 + xkl. \tag{2}$$

6 Returning to the empty stack

When the suffix is empty, only O_0 can terminate a nonempty word. Its terminal column is $d = (1, 1)^{\mathsf{T}}$. The same first-step decomposition gives completion columns Kd and $(K - I)d$ for enabled and disabled pops. A first push from empty creates control L with disabled pops, so its completion series is $k - 1$.

A nonempty complete path starts either with a bypass, or with this sole push and its first return to empty; another complete path follows. Thus

$$f = 1 + xf + x(k - 1)f = 1 + xkf. \tag{3}$$

Comparing (2) and (3) gives $l = f$, since both equal $(1 - xk)^{-1}$. Hence $u = k - f$, and the first identity in (1) reduces to

$$k = 1 + xk(4k - 2f - 1), \qquad f = 1 + xkf. \tag{4}$$

The subtraction in (4) is algebraic simplification of nonnegative path series, not a negative-weight combinatorial construction.

Using $xkf = f - 1$ in (4) yields

$$(4 - 3f)k = 1 + 2f - 2f^2.$$

The factor $4 - 3f$ has constant term 1. Substitution into (3) now gives

$$(f - 1)(4 - 3f) = xf(1 + 2f - 2f^2),$$

which expands to the polynomial in Theorem 1. The construction gives $f(0) = 1$, and the polynomial's derivative with respect to f at $(0, 1)$ is 1. Hence this constant term specifies a unique formal-power-series branch, completing the proof. □

7 Verification and scope

The independent exhaustive recognizer, basis-membership oracle, and canonical sorter agree on every input through length 8. Independently labelled forest paths match the exact permutation sets through length 6 with no duplicates; their expansions equal canonical histories. Forest and word operation languages are compared directly. The first-return series agrees with the forest recurrence through degree 12, and exact polynomial arithmetic checks the elimination. All checks run locally; they are supplementary to the all-length proof above.

A companion Lean 4.34.1 certificate proves Theorem 1 end to end. It starts from an independently defined physical stack with no bypass primitive, proves the canonicalization and the forest, word, and unambiguous grammar bijections, and constructs finite cardinalities of distinct sortable inputs. The first-return power-series identities follow from cardinality-preserving decompositions. The final checked theorem derives the cubic for that actual counting series and proves uniqueness of its constant-term-1 branch, without assuming a recurrence or a counting formula. The certificate uses a pinned mathlib revision and only the standard Lean foundational axioms. Its exact 25-module snapshot, source hashes and reproducible build accompany the paper. Formal verification applies to this unbounded enumeration theorem. A later combined 38-module certificate also proves minimum physical capacity, actual bounded permutation cardinalities, the uniform capacity recurrence, uniqueness and coefficient stabilization. The analytic pole law and Brownian-limit extension remain written results outside Lean.

The first coefficients, including the empty input, are

$$1, 1, 2, 6, 24, 116, 628, 3636, 21956, 136428, 865700, \ldots.$$

The target cubic and prior canonical rule were known before this work. The forest comparisons, word projection, and first-return equations were derived from the operations, not fitted to these numbers. An independently written PDA derivation reaches the same reduced system; detailed proofs, source hashes, and implementation evidence remain alongside this note in the repository.

References

[1] M. D. Atkinson. Generalized stack permutations. *Combinatorics, Probability and Computing* 7 (1998), 239–246. DOI. Author-hosted text.

[2] Vincent Vatter. An assortment of problems in permutation patterns: unimodality, equivalence, derangements, and sorting. Section 5, especially Problem 5.11. arXiv:2602.16355v2.

[3] M. H. Albert, C. Homberger, J. Pantone, N. Shar, and V. Vatter. Generating permutations with restricted containers. *Journal of Combinatorial Theory, Series A* 157 (2018), 205–232. arXiv:1510.00269.

[4] The On-Line Encyclopedia of Integer Sequences. A393395: permutations sortable by a $(2, 2)$-stack.

[5] M. H. Albert and M. D. Atkinson. Sorting with a forklift. *Electronic Journal of Combinatorics* 9(2) (2003), R9, Section 6. arXiv:cs/0209016.

[6] Murray Elder, Geoffrey Lee, and Andrew Rechnitzer. Permutations generated by a depth 2 stack and an infinite stack in series are algebraic. *Electronic Journal of Combinatorics* 22(2) (2015), P2.16, Section 4. DOI.